

Documentação: Fluxo do Programa App.py

Análise da Interface e Comunicação do Jogo de Truco

1. Visão Geral da Arquitetura

O arquivo **App.py** atua como o cliente gráfico (Frontend) para um jogo de Truco Paulista. Ele foi desenvolvido utilizando a biblioteca **Tkinter** para a interface com o usuário e implementa um modelo de comunicação híbrido com o **servidor_tcp.py**:

- **Sockets (TCP):** Utilizados para enviar comandos (como jogar uma carta, pedir truco) e receber dados privados do servidor (como as cartas da própria mão e notificações diretas).
- **Memória Compartilhada (SharedMemory):** Utilizada para ler o estado público do jogo (placar, cartas na mesa, de quem é a vez, e a carta "vira"). Isso permite atualizações rápidas da interface sem sobrecarregar o tráfego de rede.

2. Fluxo Principal de Execução

O ciclo de vida do aplicativo segue as seguintes etapas principais:

1. **Inicialização:** Ao abrir o programa, a classe `TrucoApp` é instanciada. O método `_build_ui()` é chamado para desenhar a mesa verde, os botões, o placar e o painel de conexão.
2. **Conexão:** O usuário insere o IP e a Porta e clica em "Conectar". O método `_thread_conectar` cria o socket e se conecta ao servidor. Após receber a mensagem de boas-vindas com o seu "ID", o cliente se acopla à Memória Compartilhada (`MesaTruco`).
3. **Loops Simultâneos:** A partir da conexão, dois ciclos passam a rodar em paralelo:
 - `_receiver_loop()` (Thread): Fica escutando as mensagens diretas do socket (recebimento de cartas, avisos de truco).
 - `_loop_jogo()` (Main Thread/Tkinter): Lê a memória compartilhada periodicamente (a cada 100ms) usando `_ler_estado()` e chama `_atualiza_ui()` para redesenhar a mesa.
4. **Interação do Jogador:** Quando é a vez do jogador, a UI é liberada. O jogador seleciona uma carta e escolhe uma ação (Jogar, Coberta ou Truco). A ação é enviada via Socket para o servidor.

3. Principais Funções e Seus Papéis

Função / Método	Descrição e Funcionamento
-----------------	---------------------------

<code>_build_ui()</code>	Responsável por instanciar todos os widgets do Tkinter. Ele estrutura a janela em áreas: o cabeçalho (placar), a área central (mesa e mão do jogador), e o painel lateral (conexão e logs).
<code>_thread_conectar(ip, porta)</code>	Executada em uma thread separada para não travar a interface. Realiza o <i>handshake</i> com o servidor via TCP. Uma vez conectado, tenta acessar o bloco de memória "MesaTruco" criado pelo servidor.
<code>_receiver_loop()</code>	Roda infinitamente aguardando dados do servidor. Ele acumula os bytes recebidos e extrai mensagens completas (usando <code>\n</code> como delimitador). Ao receber uma mensagem, encaminha para <code>_processa_mensagem()</code> .
<code>_ler_estado()</code>	Acessa a <code>SharedMemory</code> e decodifica os primeiros 1024 bytes (convertendo de bytes para uma string JSON e depois para um dicionário Python). Retorna o estado atual da rodada e mesa.
<code>_atualiza_ui(estado)</code>	Pega o dicionário lido por <code>_ler_estado()</code> e atualiza os elementos visuais. Ele muda o placar, coloca a carta Vira na tela, desenha as cartas jogadas na mesa pelos outros jogadores e bloqueia/desbloqueia os botões dependendo de quem é a vez (<code>estado["vez"] == self._meu_id</code>).
<code>_jogar()</code> / <code>_jogar_coberta()</code>	Captura o índice da carta selecionada na UI e formata uma string como "JOGAR:0" ou "JOGAR:0:COBERTA", enviando essa string pelo socket ao servidor. Após jogar, bloqueia imediatamente as próprias ações até que a vez retorne.

Atenção à Sincronização: O uso de `SharedMemory` implica que cliente e servidor devem estar na mesma máquina (mesmo sistema operacional) para que esse código funcione perfeitamente sem modificações adicionais. Para jogar em computadores distintos em uma rede LAN, o estado precisaria ser empacotado e enviado via Socket pelo servidor.

4. Integração das Classes Auxiliares

O aplicativo não possui as regras de negócio do Truco codificadas nele. As regras estão nos demais arquivos:

- **Carta e Baralho:** Responsáveis por representar os naipes, valores e a hierarquia de forças (manilhas e comparações de força considerando a carta "vira"). A UI de `App.py` apenas chama funções como `parse_carta` para transformar as strings geradas pela classe `Carta` em elementos visuais amigáveis ("🀄", "♣", "♥").
- **Mesa, Rodada e Partida:** O servidor mantém a instância dessas classes e aplica as regras. O resultado da rodada (ex: quem ganhou a queda e qual o placar) é calculado lá e exposto passivamente na `SharedMemory` para o `App.py` apenas espelhar visualmente.